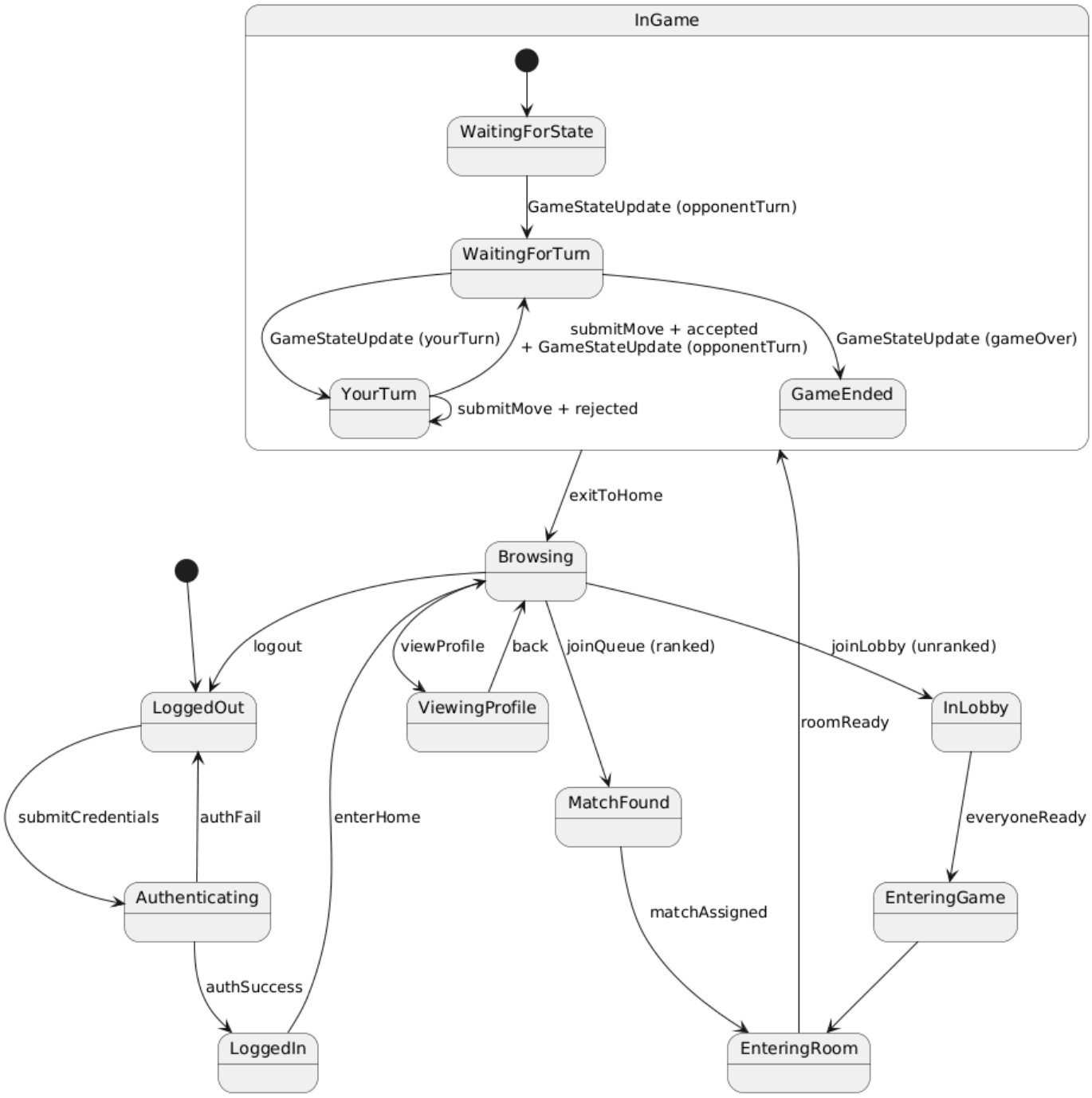
Sequence Diagrams — Overview

Complete User Flow — End to End



Sequence 04.01 — Authentication (Login / Logout / Sign Up)

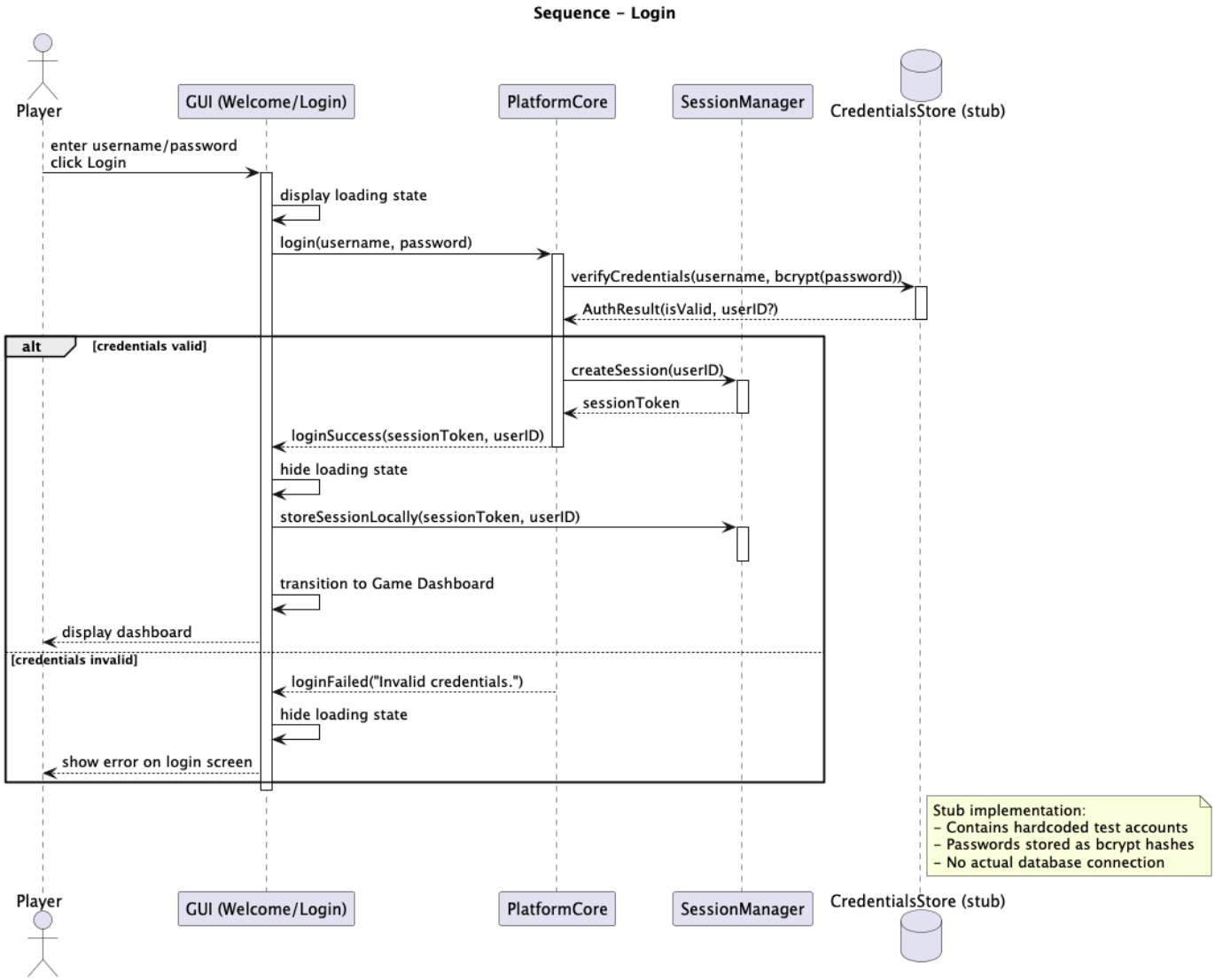
> `../04.01 Auth Sequence.md` — full descriptions and PlantUML source for all three auth flows.

Participants: Player, GUI (Welcome/Login/Register), PlatformCore, SessionManager, CredentialsStore (stub)

Login

> **Diagram:** `../04.01.01 Login Sequence.png` — shows the full login flow including the `alt` block for valid vs. invalid credentials, the `SessionManager` session creation path, and the local session storage step before the dashboard transition.

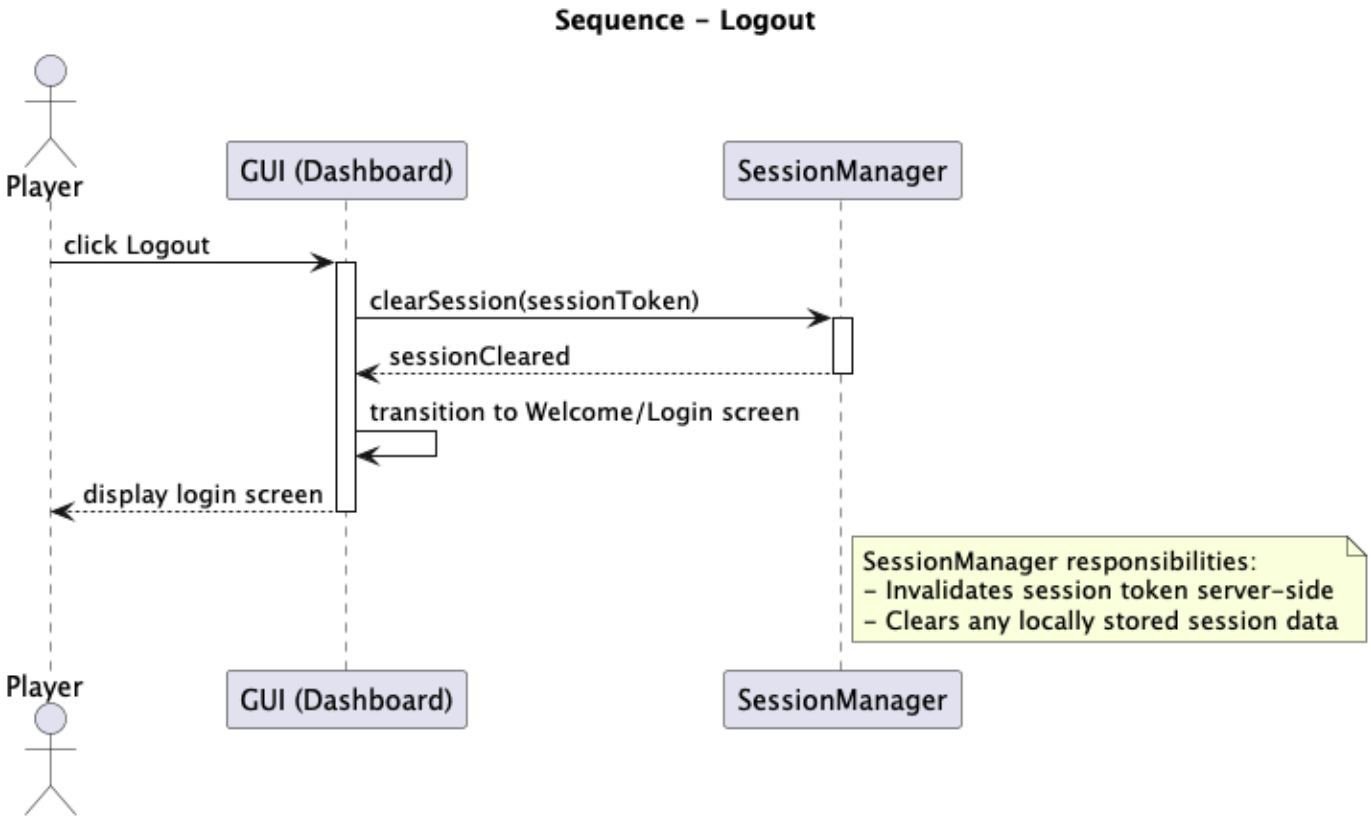
1. Player enters username + password, clicks Login
2. GUI shows loading state, calls `PlatformCore.login(username, password)`
3. PlatformCore calls `CredentialsStore.verifyCredentials(username, bcrypt(password))`
4. **[valid]** → PlatformCore asks `SessionManager.createSession(userID)` → gets `sessionToken`
   - GUI stores session locally, transitions to Game Dashboard
5. **[invalid]** → `loginFailed("Invalid credentials.")` → GUI shows error, player retries



Logout

> **Diagram:** `../04.01.02 Logout Sequence.png` — shows that logout is handled entirely by `SessionManager` with no involvement from `PlatformCore`, and that both server-side token invalidation and local session data clearance happen in the same step.

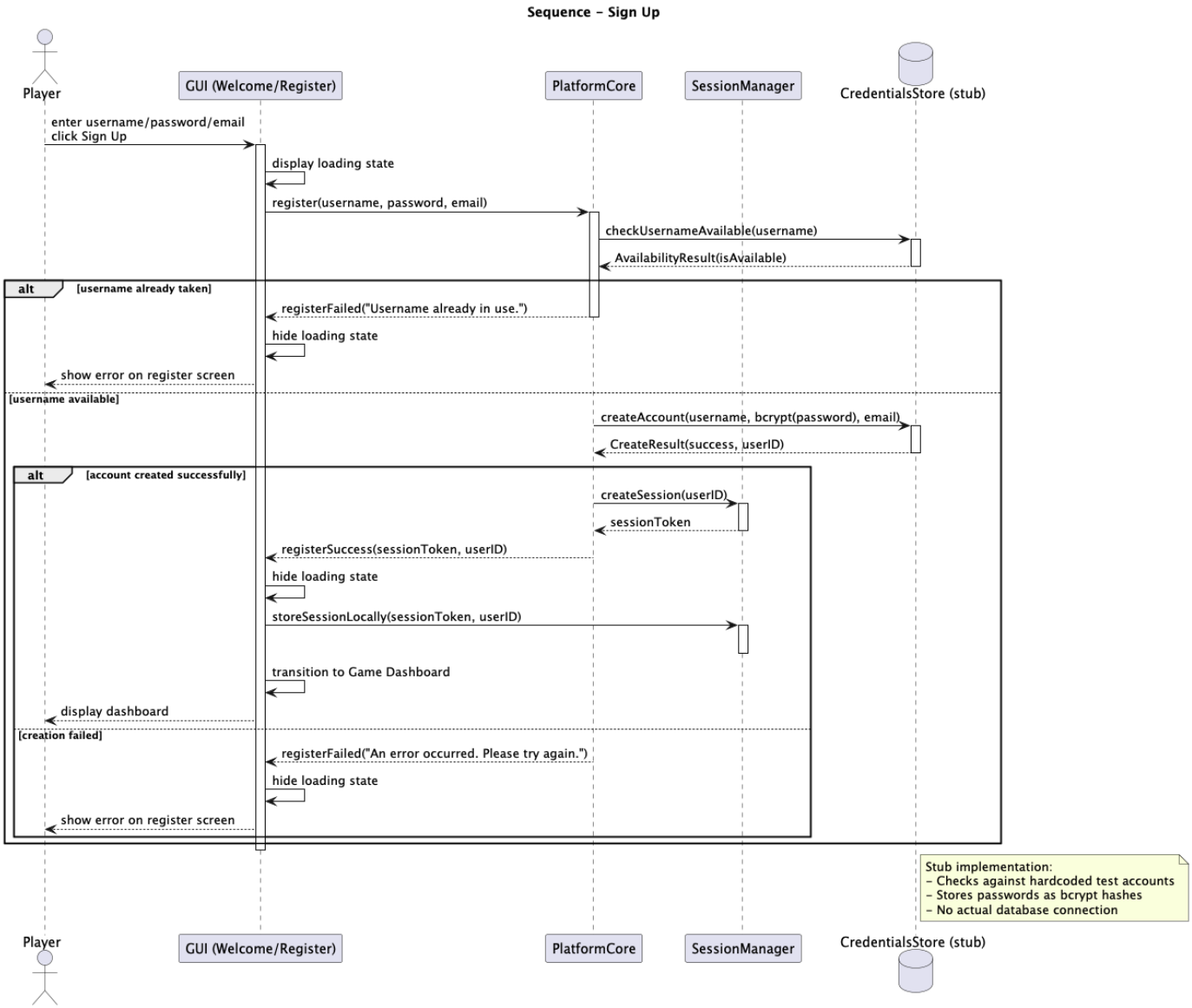
1. Player clicks Logout from Dashboard
2. GUI calls `SessionManager.clearSession(sessionToken)` directly — no PlatformCore needed
3. Session invalidated server-side + local data cleared
4. GUI transitions back to Welcome/Login screen



Sign Up

> **Diagram:** `../04.01.03 Sign In Sequence.png` — shows the two-stage `alt` block: first checking username availability, then attempting account creation, with the auto-login session creation on success.

1. Player enters username + password + email, clicks Sign Up
2. GUI calls `PlatformCore.register(username, password, email)`
3. PlatformCore checks `CredentialsStore.checkUsernameAvailable(username)`
   - **[taken]** → `registerFailed("Username already in use.")` → error shown
   - **[available]** → `CredentialsStore.createAccount(username, bcrypt(password), email)`
     - **[success]** → `SessionManager.createSession(userID)` → auto-login → Dashboard
     - **[failure]** → generic error shown



Key design note: Sign Up skips the Login step — a session is created immediately on success to reduce friction. Passwords are always stored as bcrypt hashes. The `CredentialsStore` is currently a stub with hardcoded test accounts and no live database connection.

Sequence 04.02 — Join Game (Queue → Match → Session)

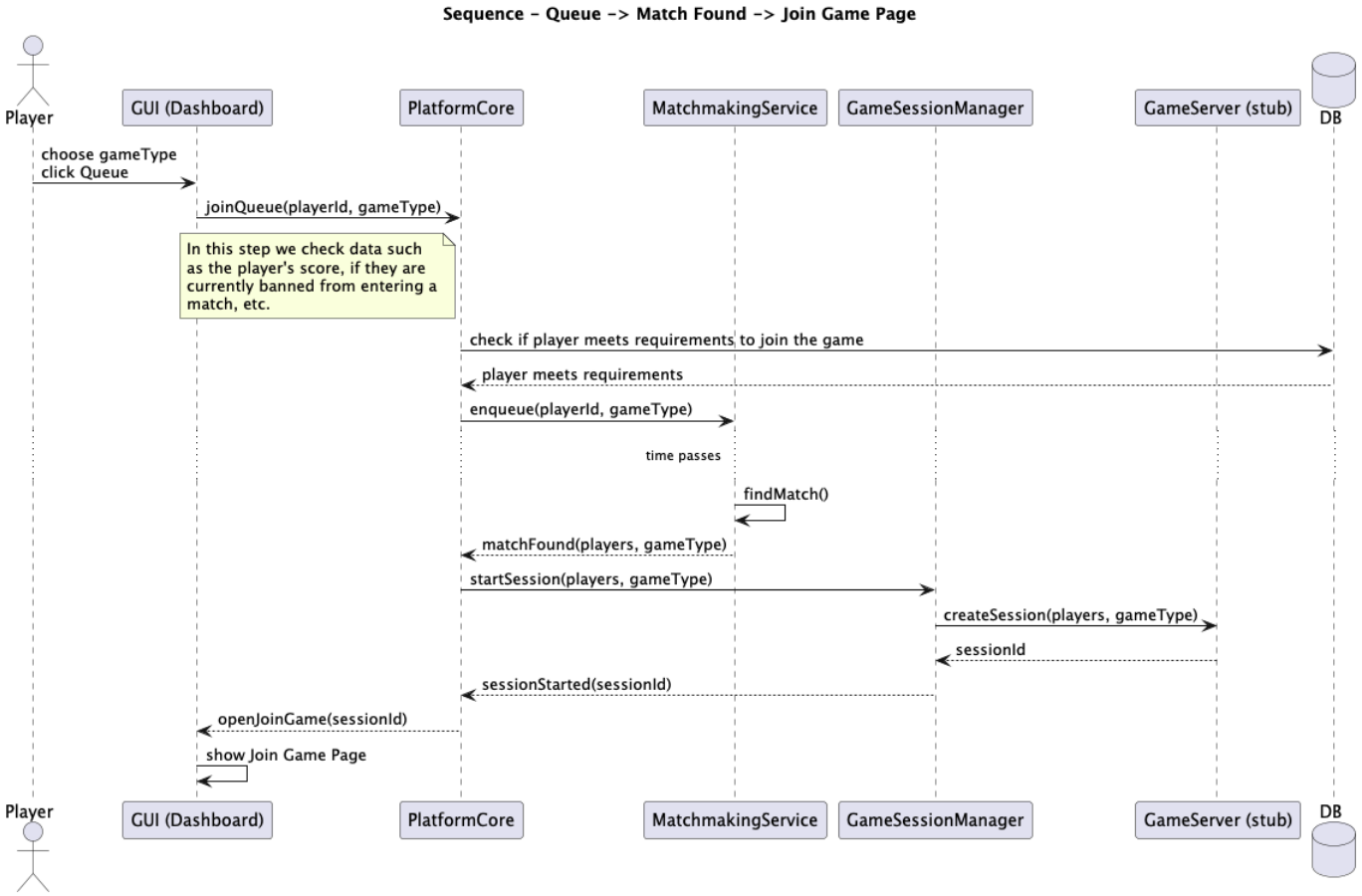
Participants: Player, GUI (Dashboard), PlatformCore, MatchmakingService, GameSessionManager, GameServer (stub), DB

Player Meets Requirements

> **Diagram:** `../04.02.01 Join Game Sequence (player meets requirements) Diagram.png` — shows the full happy path: the DB eligibility check, the `enqueue` call, the async `findMatch()` loop, and the chain from `GameSessionManager` to `GameServer` that produces a `sessionId` before the GUI is told to open the Join Game Page. The note on `PlatformCore` highlights what the requirements check covers (score, ban status, etc.).

> For the full PlantUML description, see `../04.02.01 Join Game Sequence (player meets requirements).md`.

1. Player selects a game type and clicks Queue
2. GUI calls `PlatformCore.joinQueue(playerId, gameType)`
3. PlatformCore checks DB: is the player eligible? (score check, ban check, etc.)
4. **[eligible]** → `MatchmakingService.enqueue(playerId, gameType)`
5. Time passes... `MatchmakingService.findMatch()` locates a suitable opponent
6. `MatchmakingService` → `PlatformCore.matchFound(players, gameType)`
7. `PlatformCore` → `GameSessionManager.startSession(players, gameType)`
8. `GameSessionManager` → `GameServer.createSession(players, gameType)` → receives `sessionId`
9. `PlatformCore` → `GUI.openJoinGame(sessionId)` → Join Game Page shown

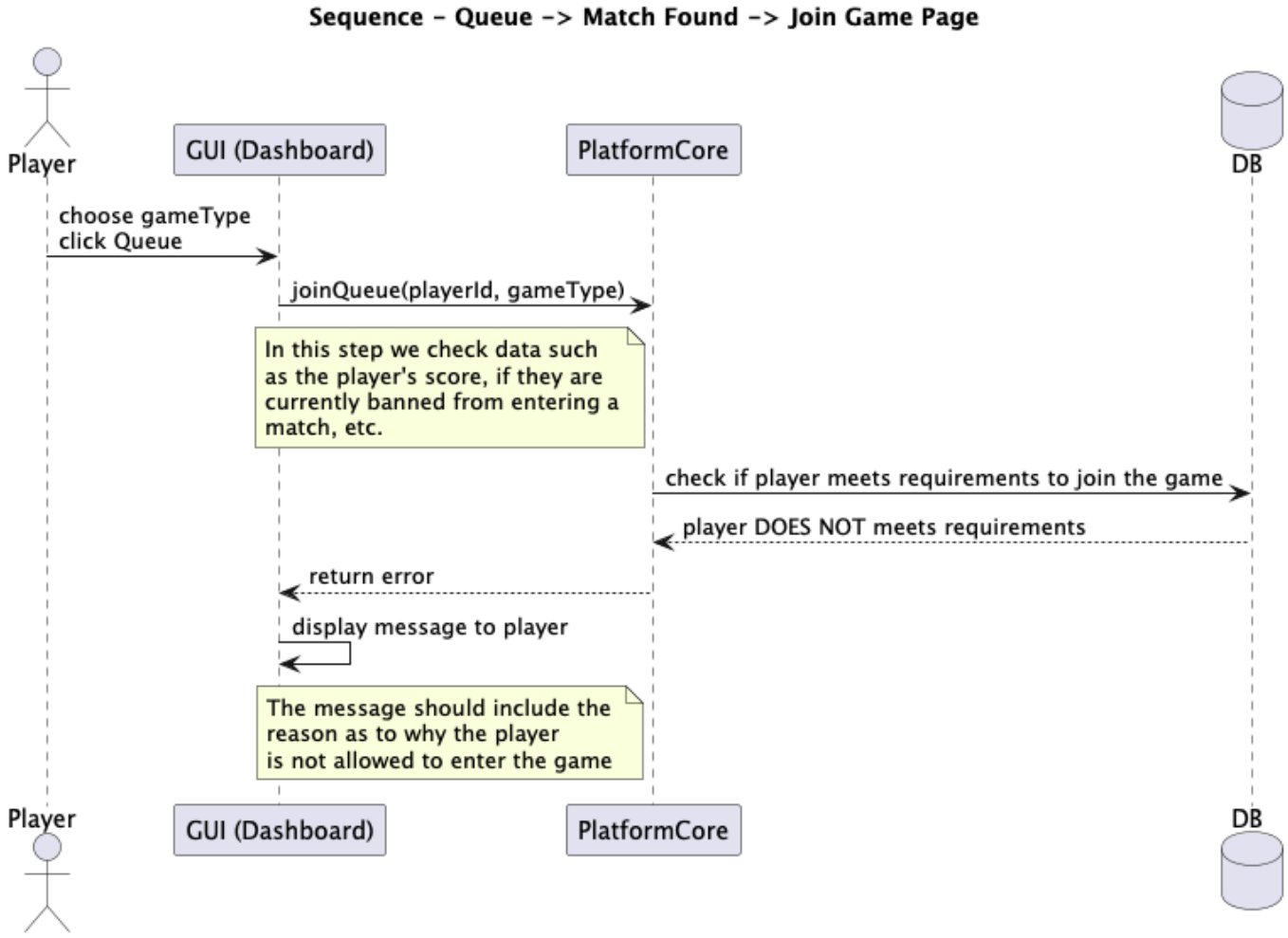


Player Does Not Meet Requirements

> **Diagram:** `../04.02.02 Join Game Sequence (player does not meet requirements) Diagram.png` — shows the short-circuit path where the DB check immediately returns a failure, `PlatformCore` returns an error to the GUI, and no matchmaking call is ever made. The note emphasises that the error message must include the specific reason.
>
> For the full PlantUML description, see `../04.02.02 Join Game Sequence (player does not meet requirements).md`.

1–3. Same as above

4. **[ineligible]** → DB returns failure → PlatformCore returns error with reason to GUI
5. GUI displays the reason to the player
6. Player remains on Dashboard



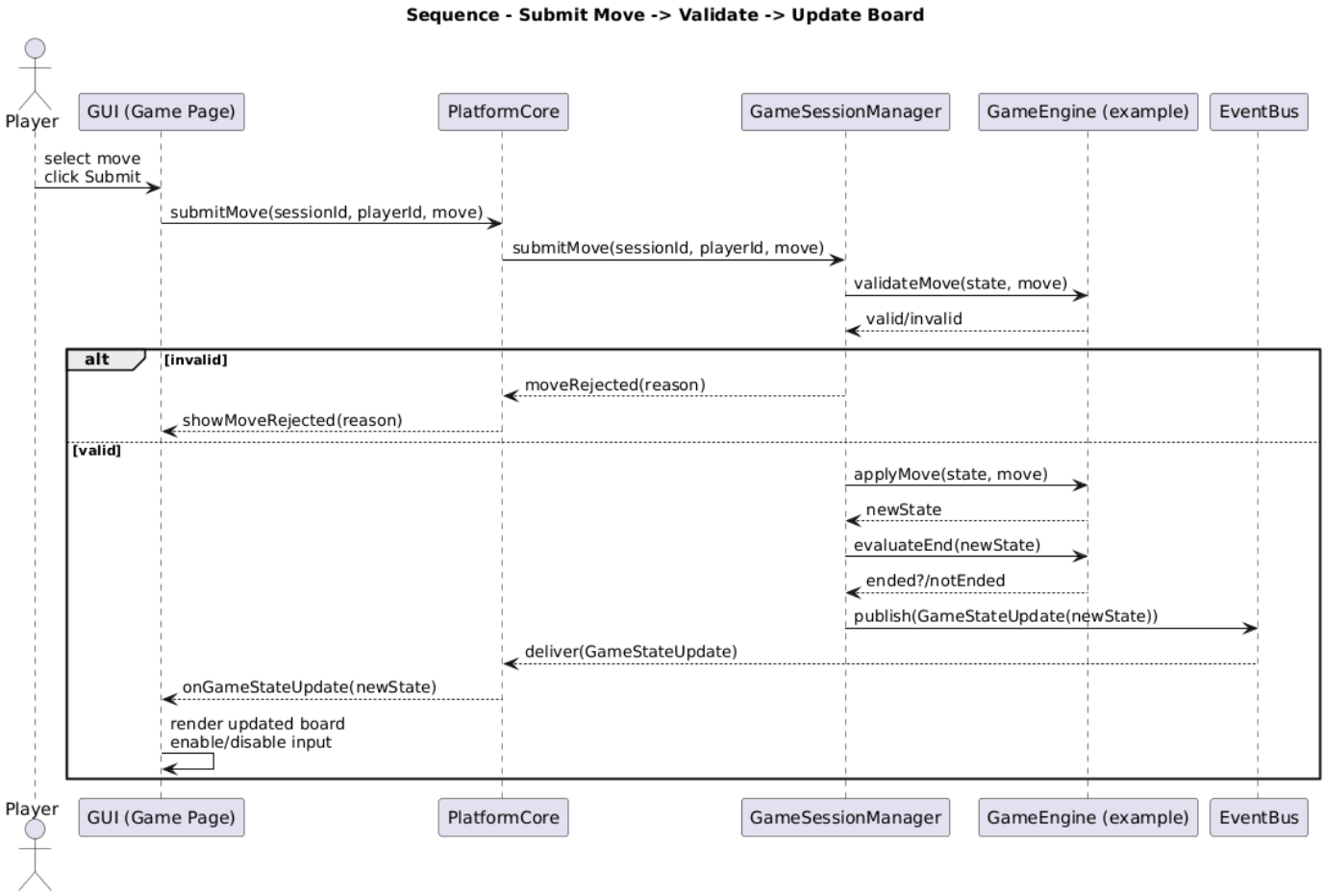
Key design note: PlatformCore acts as the eligibility gatekeeper before any matchmaking call is made. The error message must always include the reason for rejection.

Sequence 04.03 — Submit Move → Validate → Update Board

> **Diagram:** `../04.03 Submit Move Sequence Diagram.png` — shows the full move pipeline: the upfront `ack` from PlatformCore, the `validateMove` call to GameEngine, the `alt` block splitting the invalid and valid paths, the `applyMove` + `evaluateEnd` chain on the valid path, and the EventBus publish step that fans out to the GUI.
>
> For the detailed write-up and PlantUML source, see `../04.03 Submit Move Sequence.md` and `../04.03 Submit Move Sequence.puml`.

Participants: Player, GUI (Game Page), PlatformCore, GameSessionManager, GameEngine, EventBus

1. Player selects a move and clicks Submit
2. GUI calls `PlatformCore.submitMove(sessionId, playerId, move, clientStateVersion?)`
3. PlatformCore immediately sends `ack(moveReceived)` to GUI — prevents duplicate submission
4. PlatformCore forwards to `GameSessionManager.submitMove(sessionId, playerId, move)`
5. GSM calls `GameEngine.validateMove(state, move)`
   - **[invalid]** → `moveRejected(reason)` → GUI shows the rejection reason, player retries
   - **[valid]** →
     - `GameEngine.applyMove(state, move)` → returns `newState` (includes `nextTurn`)
     - `GameEngine.evaluateEnd(newState)` → checks end condition
     - **[not ended]** → `EventBus.publish(GameStateUpdate(newState))`
     - **[ended]** → `EventBus.publish(GameEnded(newState, outcome))`
6. EventBus → PlatformCore → GUI
7. GUI re-renders the board, enables/disables input based on whose turn it is



Key design note: `GameSessionManager` is the single source of truth. No state mutation occurs until `validateMove` returns valid. The optional `clientStateVersion` supports optimistic concurrency control to prevent stale move submissions.

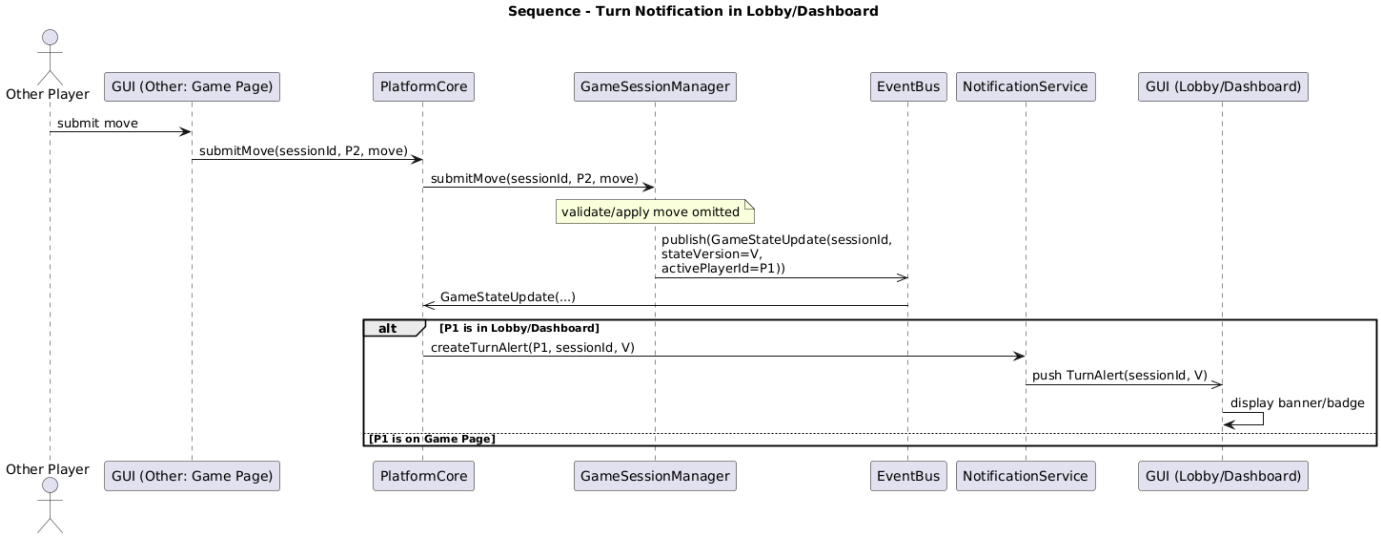
Sequence 04.04 — Turn Notification (Cross-Page Awareness)

> **Diagram:** `../04.04 Turn Notification Sequence Diagram.png` — shows the flow from Other Player's move submission through to the `alt` block that splits on P1's current UI context: the `NotificationService` push path when P1 is on the Lobby/Dashboard, versus the direct `GameStateUpdate` route when P1 is already on the Game Page. The `stateVersion=V` in the event is visible in the diagram.
>
> For the detailed write-up and PlantUML source, see `../04.04 Turn Notification Sequence.md` and `../04.04 Turn Notification Sequence.puml`.

Participants: Other Player (P2), GUI (Game Page), PlatformCore, GameSessionManager, EventBus, NotificationService, GUI (Lobby/Dashboard)

This sequence answers: *what happens when it becomes P1's turn but P1 is not currently on the game page?*

1. P2 submits a move through their Game Page
2. Move flows: `GameGUI → PlatformCore → GameSessionManager`
3. GSM validates and applies the move (details in 04.03), then publishes:
   `EventBus.publish(GameStateUpdate(sessionId, stateVersion=V, activePlayerId=P1))`
4. EventBus delivers to PlatformCore
5. PlatformCore checks P1's current UI context:
   - **[P1 is on Lobby/Dashboard]** → `NotificationService.createTurnAlert(P1, sessionId, V)` → push `TurnAlert(sessionId, V)` to P1's Lobby/Dashboard GUI → banner or badge displayed
   - **[P1 is already on Game Page]** → route `GameStateUpdate` directly to P1's GameGUI → board updates normally



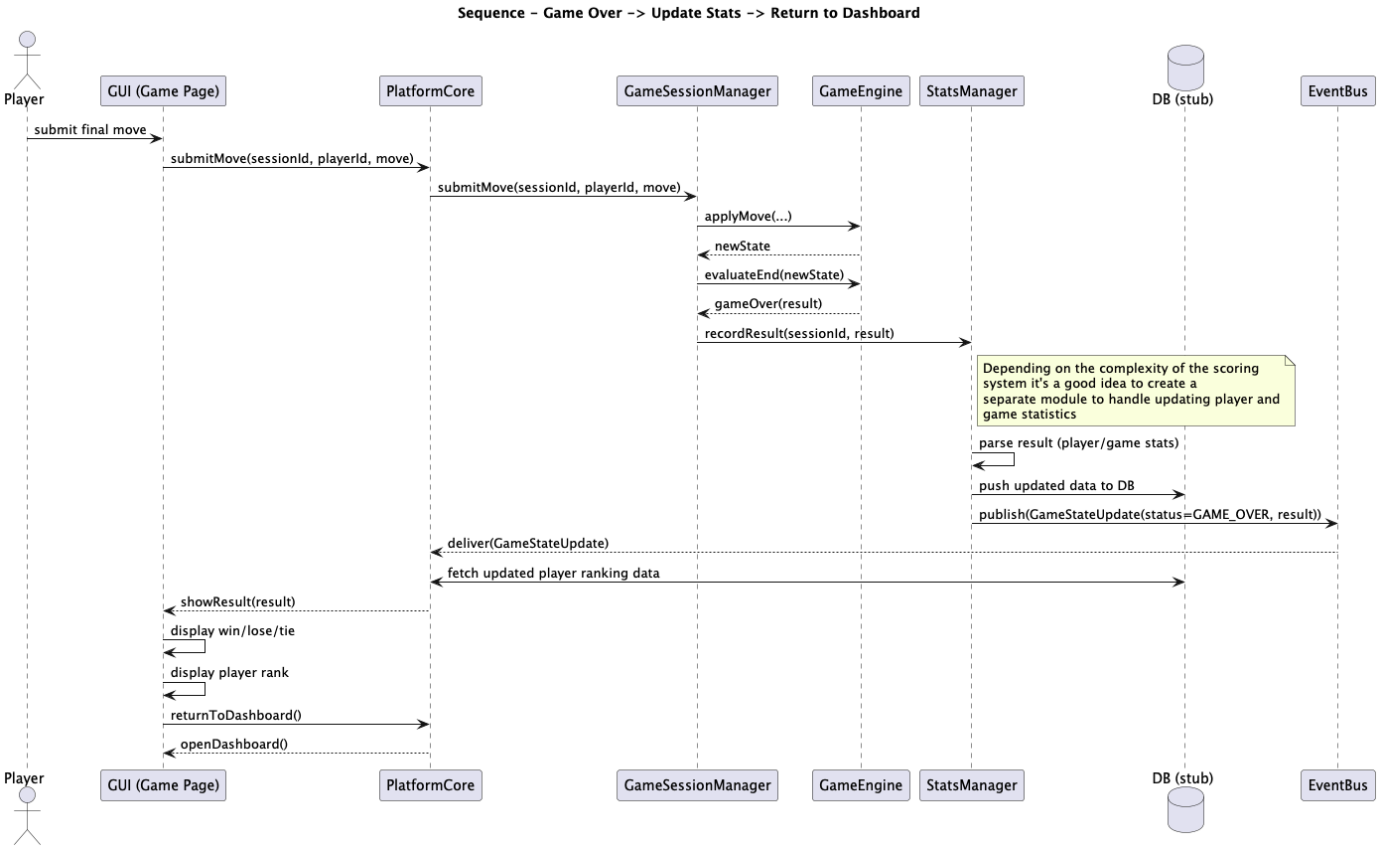
Key design note: `stateVersion` in the alert lets the UI verify it is not displaying a stale notification. PlatformCore must track each player's active UI context to route correctly and avoid duplicate alerts.

Sequence 04.05 — Game Ends → Update Stats → Return to Dashboard

> **Diagram:** `../04.05 Game Ends Sequence Diagram (updated).png` — shows how a terminal `evaluateEnd` result triggers `StatsManager.recordResult`, the DB write, and the `EventBus.publish(GameStateUpdate(status=GAME_OVER, result))` that brings the result back to PlatformCore. The diagram also shows PlatformCore fetching the updated ranking from DB before calling `showResult` on the GUI, and the final `returnToDashboard` / `openDashboard` round-trip. The note on `StatsManager` explains why it is a separate module.
>
> For the full description, see `../04.05 Game Ends Sequence.md`.

Participants: Player, GUI (Game Page), PlatformCore, GameSessionManager, GameEngine, StatsManager, DB (stub), EventBus

1. Player submits a final move
2. Move flows through the normal Submit Move pipeline (04.03)
3. `GameEngine.evaluateEnd(newState)` returns `gameOver(result)`
4. `GameSessionManager` → `StatsManager.recordResult(sessionId, result)`
5. `StatsManager` parses the result into per-player, per-game-type stats
6. `StatsManager` pushes updated data to `DB`
7. `StatsManager` → `EventBus.publish(GameStateUpdate(status=GAME_OVER, result))`
8. EventBus → PlatformCore, which also fetches updated ranking data from DB
9. PlatformCore → `GUI.showResult(result)`
   - GUI displays: win / lose / tie outcome + updated player rank
10. Player clicks return → `GUI.returnToDashboard()` → `PlatformCore.openDashboard()`



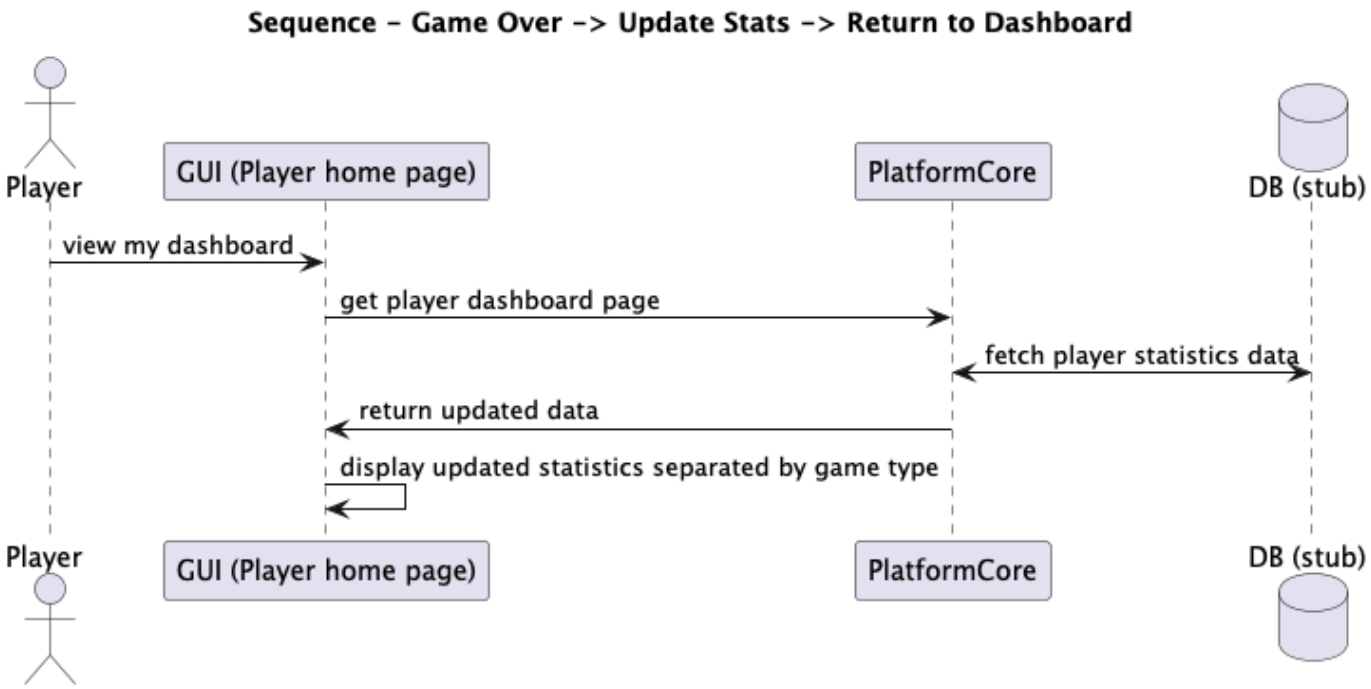
Key design note: `StatsManager` is a dedicated module separate from `GameSessionManager` because scoring and stat aggregation can grow in complexity independent of session lifecycle logic. Stats are stored per game type, enabling the per-game breakdown in sequence 04.06.

Sequence 04.06 — View Updated Player Statistics

> **Diagram:** `../04.06 Update Player Statistics Sequence Diagram.png` — shows a minimal three-participant flow: the GUI requests the dashboard page from PlatformCore, PlatformCore fetches statistics data from the DB, and the returned data is displayed on the player home page separated by game type. This diagram confirms this sequence is a pure read — no game logic or session involvement.
>
> For the full description, see `../04.06 Update Player Statistics Sequence.md` .

Participants: Player, GUI (Player Home Page), PlatformCore, DB (stub)

1. Player navigates to their home page
2. GUI calls `PlatformCore.getPlayerDashboardPage()`
3. PlatformCore fetches player statistics data from DB
4. PlatformCore returns updated data to GUI
5. GUI displays statistics separated by game type



Key design note: This is a pure read-only fetch. Stats are already computed and persisted by `StatsManager` at game-end time (04.05), so this sequence involves no game logic.

Component Interaction Summary

Component	Role Across All Sequences
`GUI`	Captures input, renders state, routes to PlatformCore; never owns logic
`PlatformCore`	Central coordinator; routes all requests; owns notification routing
`SessionManager`	Issues and invalidates session tokens; single source of session truth
`CredentialsStore` (stub)	bcrypt credential verification; username availability checks
`MatchmakingService`	Enqueues players and finds matches by game type
`GameSessionManager`	Owns authoritative game state; drives the move pipeline
`GameEngine`	Pure rule evaluation — `validateMove`, `applyMove`, `evaluateEnd`; no side effects
`EventBus`	Async pub/sub; decouples state mutation from UI updates and notifications
`NotificationService`	Generates and pushes turn alerts when the active player is off the game page
`StatsManager`	Parses game results into per-player, per-game-type stats; persists to DB
`DB` (stub)	Persistence for credentials, session data, and player statistics